Chapitre 1

Graphes

Sommaire.

1 Algorithme A^* 1

2 Algorithme de Kosaraju 2

3 Arbres couvrants minimaux. 3

4 Comptage dans un graphe biparti. 5

Les propositions marquées de ★ sont au programme de colles.

1 Algorithme A^*

Définition 1

L'idée est d'exploiter des connaissances qu'on a sur le graphe pour trouver un chemin plus rapidement.
Si on souhaite aller de Paris à Marseille, passer par Bruxelles semble être une mauvaise idée.
Pour rendre compte de cela, on cherche à biaiser la recherche en fonction de la proximité avec la destination.
On introduit donc une heuristique h qui à chaque sommet associe une valeur, croissante avec la distance.
Dans notre cas, la distance à vol d'oiseau semble pertinente. Pour un même graphe, on peut avoir plusieurs exécutions en fonction de l'heuristique.

L'algorithme A^* ressemble beaucoup à celui de Dijkstra, où l'on ajoute $h(x)$ à la valeur associée à x.
Pour chaque extraction, on privilégie le sommet minimisant $h(x) + d(s, x)$.
Il faut ajouter une contrainte à l'heuristique pour garantir que le chemin trouvé est bien le plus court.
Une heuristique h est dite **admissible** si pour tout x, $h(x) \leq d(x, t)$.

Proposition 2

Si l'heuristique est admissible, le chemin trouvé par A^* est bien le plus court.

Définition 3

Algorithme 1 : Algorithme A^*

Entrées : s, t, G, h
Sorties :
$d \leftarrow$ tableau de distance à s
$F \leftarrow$ file de priorité
$(x, y) \leftarrow (s, h(s))$
tant que $x \neq t$ **faire**
  $d[x] \leftarrow y - h(x)$
  pour chaque z voisin de x **faire**
    $w \leftarrow d[x] + p(x, z) + h(z)$
    si $z \in F$ et $w < F[z]$ **alors**
      Mettre à jour z dans F.
    fin
    sinon si $d[x] + p(x, z) < d[z]$ **alors**
      Insérer (z, w) dans F.
    fin
  fin
  $(x, y) \leftarrow$ Extraire(F).
fin
retourner y

Preuve :

Correction. On suppose que l'algorithme nous renvoie un chemin $s \to x_1 \to x_2 \to ... \to x_p \to t$ de longueur d non optimale.
Si h est admissible, alors $d(s, x) + h(x) \leq d(s, x) + d(x, t)$.
Il existe un chemin $s \to y_1 \to y_2 \to ... \to y_q \to t$ de longueur d_{OPT} strictement plus petite.
L'algorithme a extrait t avant de finir son exécution, t est le sommet de plus petit poids dans la file.
On considère y_i le premier des y_k sommets encore présent dans la file de priorité : $d(s, y_i) + h(y_i) > d$.
Or h est admissible : $d(s, y_i) + d(y_i, t) \geq d(s, y_i) + h(y_i) > d$ d'où $d_{OPT} > d$, contradiction.
Pourquoi existe-t-il un tel y_i dans F ?
Supposons qu'on ait extrait tous les y_k de F, on aura aussi mis à jour t, donc on aurait eu le chemin optimal.
Supposons que certains n'aient pas été insérés, les autres extraits. Ce n'est pas possible car chaque extraction est associée à une insertion.
Complexité. La complexité dépend fortement de l'heuristique, par son temps de calcul et sa qualité.
Il est parfois plus lent que Dijkstra.

Définition 4: Monotonie d'une heuristique.

Une heuristique est dite monotone si elle vérifie :

$$\forall x, y \quad |h(x) - h(y)| \leq d(x, y).$$

Une heuristique monotone et pour laquelle $h(t) = 0$ est admissible.

Proposition 5

Si l'heuristique est monotone, alors A^* ne traite jamais deux fois le même sommet.

Preuve :

Supposons que le sommet x soit rétiré deux fois de la file de priorité.
On a retiré x la seconde fois à cause d'une valeur inférieure à celle de la première fois.
À chaque itération, on retire le sommet minimisant $d(s, z) + h(z)$.
On peut considérer le chemin optimal menant à x : $s \to z_1 \to ... \to z_n \to x$, trouvé par l'algorithme.
Soit z_i le premier sommet encore dans la file de priorité à l'extraction de x.
La valeur associée à celle de x dans la file de priorité est inférieure à celle associée à z_i.
Donc $d[x] + h(x) < d[z_i] + h(z_i)$, or $d[z_i] = d(s, z_i)$ car les précédents ont déjà été traités.
D'où $d[x] + h(x) < d(s, z_i) + h(z_i)$, puis $d[x] \geq d(s, x) \geq d(s, z_i)$.
Enfin, $h(x) \leq h(z_i)$ puis $|h(z_i) - h(x)| > d(x, z_i)$

2 Algorithme de Kosaraju

Définition 6

Un graphe $G = (S, A)$ est dit **fortement connexe** si et seulement si :

$$\forall (s, t) \in S^2, \ s \rightsquigarrow t \text{ et } t \rightsquigarrow s \text{ existent.}$$

Définition 7

Une **composante fortement connexe** est un sous-graphe fortement connexe auquel on ne peut pas ajouter de nouveau sommet sans retirer la forte connexité.

Proposition 8

Les CFC partitionnent l'ensemble des sommets.

Preuve :

La relation $x\mathscr{R}y$ ssi il existe un chemin de x à y et y à x est une relation d'équivalence.
— $\forall x, \ x\mathscr{R}x$ immédiat.
— $\forall x, y \ x\mathscr{R}y \Longrightarrow y\mathscr{R}x$ immédiat.
— Soient $x\mathscr{R}y$ et $y\mathscr{R}z$, alors $x \rightsquigarrow y \rightsquigarrow z$ et $z \rightsquigarrow y \rightsquigarrow x$ existent.
Les classes d'équivalence de $\mathscr{R}$ partitionnent l'ensemble S, ce sont les CFC.

Proposition 9

Le graphe des CFC est acyclique.

Preuve :

Si le graphe contient un cycle, on peut alors exhiber un chemin entre deux sommets de CFC différentes, absurde.

Définition 10: Kosaraju

L'algorithme de Kosaraju répond au problème de recherche des CFC en utilisant seulement deux parcours en profondeur.

Algorithme 2 : Algorithme de Kosaraju

Entrées : Un graphe $G = (S, A)$
Sorties : Les CFC de G
1. Réaliser un parcours en profondeur sur G et noter les sommets dans l'ordre de fin de parcours.
2. Réaliser un parcours en profondeur sur le graphe transposé de G en commençant par les derniers sommets.
3. Pour chaque sommet, on note le sommet de départ du parcours en profondeur.
On obtient alors les CFC.

Correction totale.
Soient x et y deux sommets de G tels que l'algorithme les met dans la même CFC. Montrons qu'ils le sont bien.
Alors lors du second parcours, on a lancé un parcours depuis s qui a découvert x et y.
On sait que s a fini d'être traité après x et y dans le premier parcours. Donc $x \rightsquigarrow s$ et $y \rightsquigarrow s$ existent.
On utilise un parcours en profondeur, donc les dates de début et de fin forment un parenthèsage.
Deux cas : ds puis dx puis fx puis fs, OU dx puis fx puis ds puis fs.
Le deuxième cas est impossible puisqu'on aurait du traiter s avant de finir de traiter x.
Dans le premier cas, on sait qu'il existe un chemin de s à x, de même pour s et y.
On a alors $x \rightsquigarrow y$ et $y \rightsquigarrow x$, ils sont dans la même CFC.

Soient x, y dans la même CFC. Supposons que x soit le premier dans l'ordre d'extraction. Soit x n'est pas déjà traité lorsqu'il est extrait, on réalise un parcours en profondeur à partir de x. Il existe dans G un chemin de x à y dans $G^\top$. Il pourrait exister t tel que le chemin de x à y ne soit pas réalisé dans le parcours en profondeur, y a donc été traité avant x. Alors x aurait aussi déjà été traité, contradiction.
Sinon, ni x ni y ne sont racines d'un parcours en profondeur, en supposant que x est le premier traité, il existe un sommet r menant à x sur lequel on a exécuté un paroours en profondeur, et depuis x, on se ramène ainsi au cas précédent.

Lemme 11

$$a \vee b \iff (\neg b \implies a) \wedge (\neg a \implies b).$$

Lemme 12

Si G est acyclique et fini, il existe un sommet sans prédecesseur.

Preuve :

Supposons que tous les sommets aient un prédecesseur. On définit alors la suite (s_i) des prédecesseurs.
Puisque G est fini, $\exists i < j,\ s_i = s_j$, c'est un cycle.

Définition 13

Soit φ une formule logique sous forme normale conjonctive.
On construit G_φ le graphe associé à φ dont :
— Les sommets sont les littéraux de φ et leur négation.
— Les arcs sont les implications obtenues en transformant les clauses.
Si il existe un chemin de a à b, cela correspond à une implication $a \implies b$.
Si a et b appartiennent à la même CFC, on a même équivalence.
Si a et $\neg a$ appartiennent à la même CFC, φ n'est pas satisfiable.
Il faut encore montrer que si cette situation ne se présente pas, φ est satisfiable.

On commence par construire les CFC de G_φ, on teste pour x que x et $\neg x$ n'appartiennent pas à la même CFC.
On construit le graphe des CFC, on cherche à fixer à Faux les CFC sans prédécesseurs.
Si une composante fortement connexe est fixée à Vrai, toutes les composantes accessibles depuis celle-ci le seront aussi. On recommence jusqu'à ne plus avoir de sommets à fixer.
On traite les sommets selon un ordre topologique. Si un graphe est acyclique, il admet un ordre topologique.
On peut le construire en réalisant un parcours en profondeur, où on note la date de fin de traitement.

Théorème 14

Algorithme 3 : Algorithme de satisfiabilité

Entrées : Une formule φ sous forme normale conjonctive
Sorties : Vrai si φ est satisfiable, Faux sinon
1. Construire G_φ
2. Construire les CFC de G_φ
3. Construire G' le graphe des CFC
4. Trier topologiquement G'
5. Fixer la valuation en suivant l'ordre topologique

Complexité.
On pose n le nombre de variables et m le nombre de clauses.
La construction de G_φ est en $O(n+m)$.
La construction des CFC est en $O(n+m)$.
La construction de G' est en $O(n+m)$.
Le tri topologique est en $O(n+m)$.
La fixation de la valuation est en $O(n)$.
La complexité totale est en $O(n+m)$.

3 Arbres couvrants minimaux.

Définition 15

Étant donné un graphe G non orienté, connexe et pondéré, comment relier tous les sommets en minimisant le poids total ?

Définition 16

Dans le cas des graphe, un arbre est un graphe connexe et acyclique.
On peut enraciner un tel arbre en orientant les arêtes depuis la racine.
— Tout graphe connexe a au moins $|S|-1$ arêtes.
— Tout graphe acyclique a au plus $|S|-1$ arêtes.
— Tout arbre a exactement $|S|-1$ arêtes.
On dit qu'un arbre est couvrant si tous les sommets sont extrêmités d'une arête de l'arbre.

Preuve :

On raisonne par récurrence sur la taille de $|S|$.
Initialisation. Si $|S| = 1$, c'est fini.
Hérédité. On suppose que c'est vrai pour $|S| = n$. Soit un graphe de taille $n+1$.
Si il y a un sommet de degré 1, on peut le retirer, et appliquer l'hypothèse de récurrence.
Sinon, tous les sommets sont de degré au moins 2. Alors $\sum d(s) = 2|A| \geq 2|S|$ donc $A \geq |S|-1$.

Proposition 17

Si tous les poids sont positifs, la solution à ntore problème est un arbre couvrant.

Preuve :

Si le graphe contenait un cycle, on pourrait obtenir un sous-graphe plus petit en retirant une arête du cycle.

Définition 18

Algorithme 4 : de Kruskal.

Arbre $\leftarrow \varnothing$.
Considérer les arêtes dans l'ordre croissant des poids.
pour chaque $a \in A$ **faire**
  si elle ne crée pas de cycle **alors**
    Arbre $\leftarrow$ Arbre $\cup \{a\}$.
  fin
fin
retourner Arbre

Cet algorithme renvoie une solution optimale pour le problème.
Correction.
1. L'algorithme renvoie un arbre couvrant.
2. L'arbre couvrant renvoyé est de poids minimal.

$\boxed{1.}$ Puisqu'on s'assure que l'insertion ne crée pas de cycle, on sait que le graphe renvoyé est acyclique.
Si G est connexe, pour tout $x, y \in S$ il existe un chemin entre x et y. Si x et y n'étaient pas reliés dans le graphe renvoyé, il y aurait contradiction, puisqu'on aurait pu ajouter une arête du chemin sans créer de cycle.
$\boxed{2.}$ On prouve l'invariant suivant : « $\exists \text{Arbre}_{\text{OPT}}$ couvrant minimal : Arbre $\subset \text{Arbre}_{\text{OPT}}$ ».
C'est vrai avant la boucle car $\varnothing$ est inclus dans tous les arbres optimaux. Supposons l'invariant vrai en début d'itération. Soit A_0 l'arbre optimal correspondant. On a deux cas :
— a crée un cycle, alors l'invariant est préservé car Arbre n'est pas modifié.
— a ne crée pas de cycle, alors si $a \in A_0$, c'est bon. Sinon, on cherche A_1 arbre couvrant minimal. On peut modifier A_0 pour obtenir A_1. Il existe donc un cycle contenant $a = (x, y)$, et donc a' reliant x et y. Cet arc n'a pas encore été considéré donc $p(a') \geq p(a)$. Ainsi, $A_0 \cup \{a\} \setminus \{a'\}$ est couvrant et de poids inférieur ou égal à A_0. L'invariant est préservé.
Complexité.
Listes d'adjacence. Tester la présence d'un cycle : $O(|S|)$; insérer : $O(1)$ donc en totalité : $O(|A||S|)$.
Parcours. On fait un parcours en profondeur pour mettre dans une même classe d'équivalence les sommets d'une composante connexe. Alors Tester est en $O(1)$, insérer nécessite un parcours en profondeur : $O(|S|)$, donc au total $O(|A|\log(A) + |S|^2)$.

Définition 19: Union-find.

La première opération sera : trouver à quel ensemble appartient un élément (étiquette).
On veut que deux éléments soient dans le même sous-ensemble ssi ils ont la même racine.
Trouver le sous-ensemble auquel appartient un élémént consiste à remonter à la racine.
Unir deux sous-ensembles consiste à rajouter une arête d'un représentant vers l'autre.
Alors Tester est en $O(|S|)$ et insérer est en $O(|S|)$.
On va chercher à équilibrer l'arbre en minimisant la hauteur, ainsi la hauteur des arbres augment de 1 uniquement lorsqu'ils étaient de même hauteur.

Proposition 20

Pour obtenir un arbre de hauteur h, il en faut deux de hauteur $h-1$, donc 4 de hauteur $h-2$...
Les arbres ainsi construits sont de hauteur logarithmique en leur nombre de noeuds.

Preuve :

Soit $\mathcal{P}_n$: « un arbre à n noeuds a pour hauteur $h = \log_2 n$ ».
Initialisation. La propriété est vraie avant une quelconque union, comme tous les arbres sont de hauteur 0 et ont 1 élément.
Hérédité. Soient A_1 et A_2 de n_1 et n_2 noeuds. Supposons $h_1 \leq \log_2 n_1$ et $h_2 \leq \log_2 n_2$.
Supposons sans perte de généralité que $h_1 \leq h_2$.
• Si $h_2 > h_1$, alors l'union donne un arbre A_3 de hauteur h_3 à n_3 éléments.
On a $h_3 = h_2$ car $h_1 + 1 \leq h_2$, et $n_3 = n_2 + 1$ donc $h_3 = h_2 \leq \log_2 n_2 \leq \log_2(n_2 + n_1) \leq \log_2 n_3$.
• Si $h_2 = h_1$, alors $n_3 = n_2 + n_1$ et $h_3 = h_1 + 1 = h_2 + 1$.
Supposons que $n_2 \leq n_2$ sans perte de généralité, alors $2n_1 \leq n_1 + n_2 \leq 2n_2$.
On a alors $1 + \log_2(n_1) \leq \log_2(n_3) \leq 1 + \log_2(n_2)$ puis $1 + h_3 \leq \log_2 n_3 \leq 1 + h_2$
Par antisymétrie, $\log_2(n_3) = h_3$.
Dans tous les cas, $\mathcal{P}_n$ est vraie.

Définition 21

Avec cette modification; il ne faut plus que $\log_2 n$ opérations pour remonter à la racine d'un arbre. On peut donc réaliser l'union en $\log_2 |S|$ opérations. Si on veut être efficace, il faut un tableau qui à chaque sommet associe sa hauteur.
Le coût du test test donc en $O(\log |S|)$, et de l'union en $O(1)$.
On obtient alors une complexité en $O(|A| \log |A|)$, il n'y a pas besoin de mieux pour notre structure de donnée car la complexité est limitée par le tri.

Définition 22

On appelle $\log^*$ (logarithme itéré) la fonction qui compte le nombre de fois qu'il faut appliquer log pour obtenir moins de 1.

4 Comptage dans un graphe biparti.

Définition 23

Un ensemble d'arêtes C est un couplage si pour toutes arêtes $\{a_1, b_1\}$ et $\{a_2, b_2\}$ distinctes, $\{a_1, b_1\} \cap \{a_2, b_2\} = \varnothing$.
Un graphe est biparti si on peut partitionner S en A et B tels qu'il n'y ait pas d'arêtes entre deux sommets de A et entre deux sommets de B.

Définition 24: Problème.

Étant donné un graphe biparti $G = (X \cup Y, A)$, on cherche un couplage de cardinal maximal inclus dans G.

Définition 25: Approche naïve.

L'approche naïve suivante ne marche pas :

Entrées : $G = (X \cup Y, A)$
$C \leftarrow \varnothing$.
tant que $A \neq \varnothing$ **faire**
  Soit $\{x, y\} \in A$.
  $C \leftarrow C \cup \{x, y\}$.
  Retirer de A toutes les arêtes ayant x ou y comme extrêmité.
fin
retourner C.

En effet, le couplage renvoyé n'est pas maximal (au sens de l'inclusion). On dit qu'un couplage est parfait si tous les sommets sont extrêmités d'un arête du couplage.

Définition 26

Un chemin alternant est un chemin dans G qui alterne entre des arêtes appartenant à C et des arêtes n'appartenant pas à C.
Un chemin est améliorant s'il est à la fois alternant et si ses deux extrêmités ne sont pas extrêmités d'une arête de C.

Théorème 27: de Berge.

Un couplage est maximum ssi il n'admet pas de chemin améliorant.

Preuve :

Soit C un couplage qui admet un chemin C_h améliorant.
$\boxed{\Longrightarrow}$ Par contraposée, soit C' le couplage défini par $C' = (C \setminus C_h) \cup (C_h \setminus C)$.
Il faut montrer que C' est effectivement un couplage et $|C'| > |C|$.
On a que C_h contient n arêtes de C et $n+1$ arêtes n'appartenant pas à C, donc $|C'| = |C| + 1$.
Soit x ayant un arête incidente dans C'.
– Si le chemin de passe pas par x, rien n'a changé.
– Sinon, x est soit une extrêmité du chemin, soit n'en est pas une.
— Si c'est une extrêmité du chemin, x est extrêmité d'une arête de $C_h \setminus C$, mais pas de $C \setminus C_h$.
— Sinon, x est extrêmité d'une arête de $C_h \setminus C$, mais pas de $C \setminus C_h$.
Alors C' est bien un couplage.
$\boxed{\Longleftarrow}$ Soit C un couplage et C' un couplage de cardinal plus élevé.
On va considérer $C \setminus C' \cup C' \setminus C$, composé de sommets isolés, de chemins simples, de cycles de longueurs paires.
Chaque sommet est de degré au plus 2, si une composante contient un sommet de degré 0, c'est un sommet isolé.
Si une composante contient un sommet de degré 1, c'est un chemin simple puisqu'on peut réaliser un parcours depuis celui-ci pour construire le chemin.
Si tous les sommets sont de degré 2, c'est un cycle, il alterne entre des arêtes de C et C', il est donc de taille paire.
On a $|C'| > |C|$ donc il existe une composante qui contient plus d'arêtes de C' que de C dans $C' \setminus C \cup C \setminus C'$.
Il commence et termine par une arête de C', c'est un chemin améliorant. Les extrêmités de sont pas dans C, sinon le chemin ne serait pas une composante connexe.

Définition 28

Algorithme 5 : Couplage maximal.

Entrées : Graphe biparti
$C \leftarrow \varnothing$.
tant que il existe C_h chemin améliorant **faire**
  $C \leftarrow C \setminus C_h \cup C_h \setminus C$
fin
retourner C

Définition 29

Comment trouver un chemin améliorant ?
On part d'un sommet qui n'est pas extrêmité d'une arête de C.
On réalise un parcours en profondeur où l'on impose d'alterner entre arêtes de C et de $A \setminus C$.
On s'arrête si on trouve un sommet qui n'est pas extrêmité d'une arête de C.

Algorithme 6 : Parcours

Entrées : G, deja_vus, s, b, C
deja_vus[s]←Vrai.
si s n'est pas extrêmité d'une arête de C **alors**
  retourner Vrai
fin
pour chaque voisin t de s **faire**
  si (non deja_vus[t]) et (b et $(s,t) \notin C$) ou (non b et $(s,t) \in C$) **alors**
    si Parcours(G, deja_vus, t, non b, C) **alors**
      retourner Vrai
    fin
  fin
fin
retourner Faux

Où b est Vrai ssi la dernière arête empruntée appartenait à C.
Pour retenir le chemin trouvé, on peut ajouter un tableau qui retient les successeurs, on ajouterait succ[s]← t avant l'appel.
Pour trouver une extrêmité correcte, on exécute le parcours depuis chaque extrêmité possible.
Complexité.
C étant représentée sous forme d'un tableau de listes d'adjacence.
Parcours en profondeur : $O(|S| + |A|)$, la taille de C_h est bornée par $|A|$, la modification de C est en $O(|S|)$.
On fait au plus $\frac{|S|}{2}$ itérations, à chaque itération, deux sommets de plus sont extrêmités d'une arête de C.
La complexité de la recherche d'un couplage maximum est donc ici en $O(|S|(|S| + |A|))$.